Projeto 2: Urna eletrônica

Entrega 1: Modelagem

Universidade Federal Rural de Pernambuco – UABJ · Engenharia da Computação

Modelagem e Programação Orientada a Objetos

Identificação do projeto

Campo	Informação
Título do projeto	Projeto 2: Urna eletrônica
Semestre	2026.2
Professor	Michael Oliveira da Cruz
Entrega	1. Modelagem (tag no Git: Entrega 1: Modelagem)
Data de entrega	12/11/2026

Integrantes da equipe

Nº	Nome completo	Matrícula	E-mail
1	Julia Gabriela de Andrade Barboza	20250029167	julia.barboza@ufrpe.br
2	Raiane Mayra Freitas Cintra	20250031835	raiane.cintra@ufrpe.br
3	Maria Luisa Marcelina de Meirelles	20250029238	luisameirelles7@gmail.com
4	Maria Helena da Silva Martins	20250023619	mariahelenahellena124@gmail.com

Apresentação do projeto

Descrição do problema

O sistema simula uma eleição por meio de uma urna eletrônica. Os eleitores votam em candidatos usando a urna, digitando o número do candidato e confirmando, ou votando em branco. Cada eleitor pode votar apenas uma vez e o voto é secreto: depois de votar, ninguém consegue saber em quem a pessoa votou. Quando a votação termina, a urna é encerrada, conta os votos de cada candidato, os brancos e os nulos, e gera um boletim de urna em arquivo TXT. O sistema é voltado a quem organiza e simula a eleição (mesário ou administrador) e aos eleitores cadastrados.

Tecnologias utilizadas

Item	Utilizado? (sim/não) e detalhes
Versão do Java e ferramentas de build	Sim. Java 17 (versão LTS e a mínima exigida no enunciado). Build com Maven, que baixa o driver do SQLite (org.xerial:sqlite-jdbc) e padroniza a estrutura de pastas do projeto. IDE: VS Code. Testes: JUnit 5, para conferir as regras do modelo (voto único por eleitor, urna encerrada recusando voto, voto sem ligação com o eleitor). Apoio de IA: Claude (Anthropic), usado para tirar dúvidas, revisar a modelagem e sugerir trechos de código, sempre lidos, testados e ajustados pela equipe.
Controle de versão	Sim. Git, com repositório compartilhado pela equipe, commits identificados pelos códigos de requisitos (ex.: RF2.3) e uma tag por entrega (entrega1 a entrega4).
Interface gráfica (Swing) ou menu de console	Sim, os dois, em etapas diferentes. Console na Entrega 2, só para demonstrar o domínio funcionando (votar, votar em branco, encerrar, contabilizar e tratar uma exceção). Swing na Entrega 4, com as três telas: Identificação (título ou CPF), Votação (número, candidato na tela e botões Confirma, Corrige e Branco) e Apuração (votos, brancos, nulos, total e botão de gerar o boletim). Erros aparecem em caixa de mensagem (JOptionPane) e o programa continua.
Banco de dados (SQLite / JDBC)	Sim, a partir da Entrega 3. SQLite acessado por JDBC, num único arquivo de banco. Tabelas eleitor, candidato, urna, urna_candidato e voto, criadas sozinhas na primeira execução. Uma classe DAO para cada tabela, que captura a SQLException e lança uma exceção do próprio sistema, assim a tela não sabe nada de SQL. Marcar ja_votou e gravar o voto são feitos numa só transação JDBC (commit se tudo der certo, rollback se algo falhar), para cumprir a RN2.5. A tabela voto não tem coluna ligada ao eleitor (RN2.2).
Arquivos (CSV, JSON, serialização, TXT)	Sim, só TXT. O boletim de urna é gerado em arquivo TXT pelo botão GERAR BOLETIM, disponível só com a urna encerrada, com a urna, a data, o total de cada candidato, os brancos, os nulos e o total geral. Não usamos CSV, JSON nem serialização.
Organização em pacotes	Sim. modelo (Pessoa, Eleitor, Candidato, Urna, Voto, TipoVoto e as exceções), dao (acesso ao SQLite e gravação do boletim) e gui ou app (telas e menu). O modelo nunca usa nada de tela.

Requisitos funcionais

ID	Requisito	Prioridade (alta/média/baixa)
RF2.1	Cadastrar eleitores e candidatos	Alta
RF2.2	Cadastrar urnas e dizer quais candidatos concorrem em cada uma	Alta
RF2.3	Votar digitando o número do candidato e confirmando	Alta
RF2.4	Aceitar voto em branco; número que não existe vira voto nulo, com aviso	Alta
RF2.5	Encerrar a urna e contar os votos, os brancos e os nulos	Alta
RF2.6	Gerar o boletim de urna em TXT	Média
RF2.7	(extra) Mostrar porcentagem de cada candidato e o vencedor	Baixa

Regras de negócio que o sistema deve cumprir:

ID	Regra
RN2.1	Cada eleitor vota no máximo uma vez; quem controla é o jaVotou do eleitor
RN2.2	Voto secreto: nenhuma ligação entre voto e eleitor
RN2.3	Urna encerrada não aceita voto
RN2.4	O voto é criado dentro da urna, por votar()
RN2.5	Marcar jaVotou e gravar o voto acontecem juntos
RN2.6	Só eleitor cadastrado vota; só candidato ligado à urna aparece nela

Entrega 1: Modelagem

1.1 Diagrama de classes UML

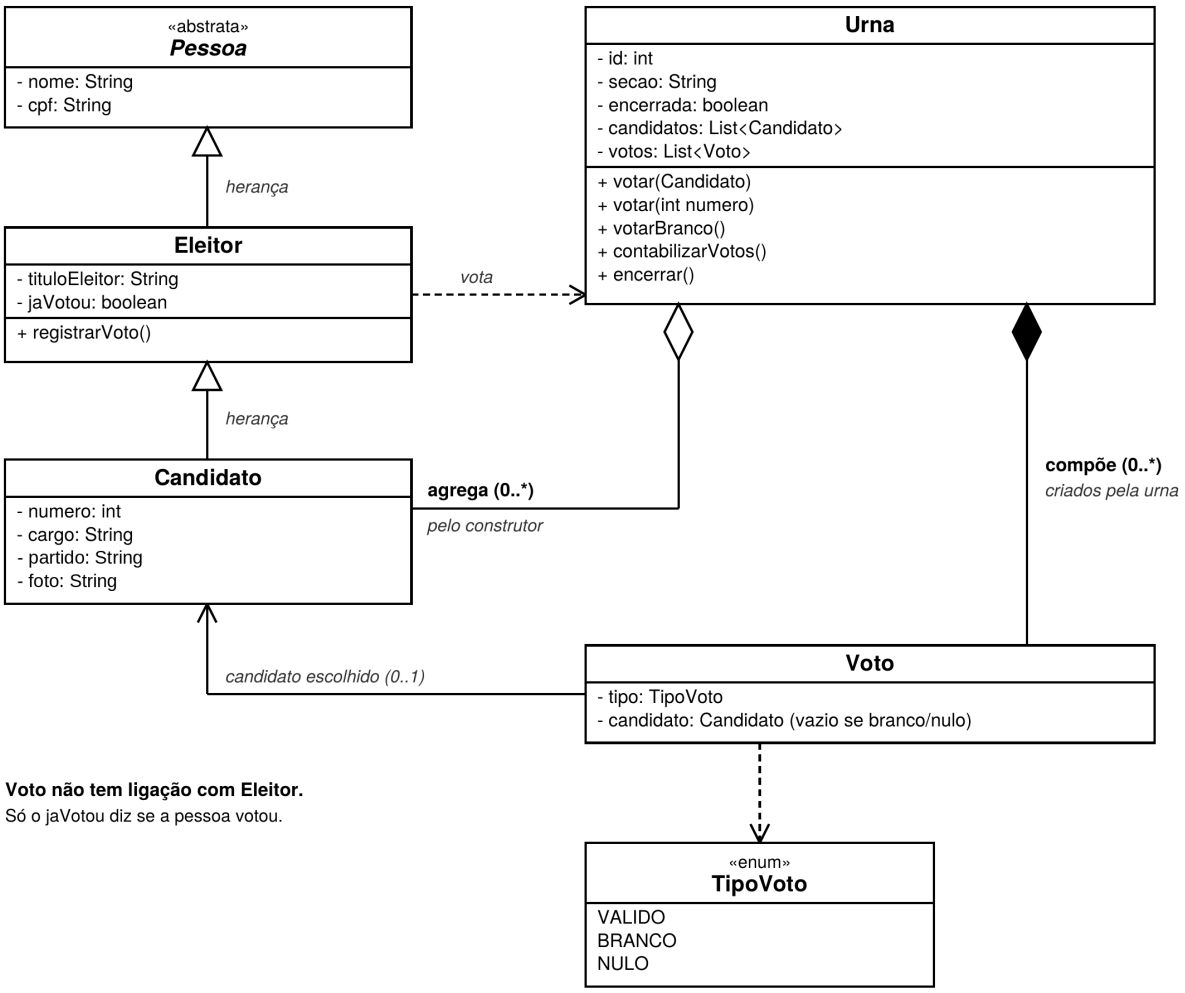


Figura 1 – Diagrama de classes da urna eletrônica.

Legenda: triângulo = herança, losango vazio = agregação, losango preenchido = composição, tracejado = uso.

Classes do diagrama, com atributos, métodos e multiplicidades:

Classe	Atributos	Métodos principais
«abstrata» Pessoa	- nome: String; - cpf: String	
Eleitor (herda de Pessoa)	- tituloEleitor: String; - jaVotou: boolean	+ registrarVoto()
Candidato (herda de Eleitor)	- numero: int; - cargo: String; - partido: String; - foto: String (caminho da imagem)	
Urna	- id: int; - secao: String; - encerrada: boolean; - candidatos: List<Candidato>; - votos: List<Voto>	+ votar(Candidato); + votar(int numero); + votarBranco(); + contabilizarVotos(); + encerrar()

Classe	Atributos	Métodos principais
Voto	- tipo: TipoVoto; - candidato: Candidato (vazio se branco ou nulo)	
«enum» TipoVoto	VALIDO, BRANCO, NULO	

Multiplicidades: Urna agrega 0..* Candidato; Urna compõe 0..* Voto; Voto aponta para 0..1 Candidato. O Voto não tem ligação com o Eleitor; só o campo jaVotou diz se a pessoa votou.

1.2 Justificativa dos relacionamentos

Classes envolvidas	Tipo	Justificativa
Pessoa e Eleitor	Herança	Todo eleitor é uma pessoa. Eleitor reaproveita nome e CPF de Pessoa e acrescenta título de eleitor e jaVotou.
Eleitor e Candidato	Herança	Todo candidato também é eleitor. Por isso a herança tem três níveis: Pessoa, Eleitor e Candidato, com super(...) preenchendo os dados da classe de cima.
Urna e Candidato	Agregação (0..*)	O candidato existe sem a urna e pode concorrer em várias urnas. A lista chega pronta pelo construtor da urna. Se a urna for apagada, os candidatos continuam existindo.
Urna e Voto	Composição (0..*)	O voto só existe dentro da urna. A lista de votos é criada na própria urna e quem cria cada Voto é a urna, nos métodos votar(). Se a urna for apagada, os votos vão junto.
Voto e Candidato	Associação (0..1)	O voto válido aponta para o candidato escolhido. Em voto branco ou nulo, não aponta para nenhum.
Voto e TipoVoto	Associação (uso do enum)	O tipo do voto (VALIDO, BRANCO ou NULO) é um conjunto fixo de valores, por isso é um enum.
Eleitor e Urna	Associação de uso (dependência, tracejado)	O eleitor vota na urna, mas a urna não guarda o eleitor. O Voto não tem ligação com Eleitor, para o voto ser secreto (RN2.2). O controle de quem já votou fica só no jaVotou, que muda por um método do próprio eleitor.

1.3 Modelo de dados ou formato dos arquivos

Banco de dados SQLite, com as tabelas abaixo:

Tabela	Colunas
eleitor	id (PK), nome, cpf, titulo, ja_votou
candidato	id (PK), eleitor_id (FK), numero, cargo, partido, foto
urna	id (PK), secao, encerrada
urna_candidato	urna_id (FK), candidato_id (FK)
voto	id (PK), urna_id (FK), tipo (VALIDO, BRANCO ou NULO), candidato_id (FK, vazio se for branco ou nulo)

Observações:

- A tabela voto não tem nenhuma coluna ligada ao eleitor, então não dá para saber em quem cada pessoa votou (RN2.2).
- A tabela urna_candidato liga as urnas com os candidatos (agregação).
- O candidato é um eleitor, por isso a tabela candidato aponta para eleitor.
- Marcar ja_votou e gravar o voto acontecem na mesma transação (RN2.5).

Arquivo TXT (boletim de urna): contém a urna, a data, o total de cada candidato, os brancos, os nulos e o total geral.

Exceções previstas:

Exceção	Quando é lançada
EleitorJaVotouException	O eleitor tentou votar de novo
UrnaEncerradaException	Tentou votar numa urna encerrada
EleitorNaoEncontradoException	Título ou CPF não cadastrado
CandidatoInvalidoException	O candidato não pertence à urna

1.4 Esboço das telas

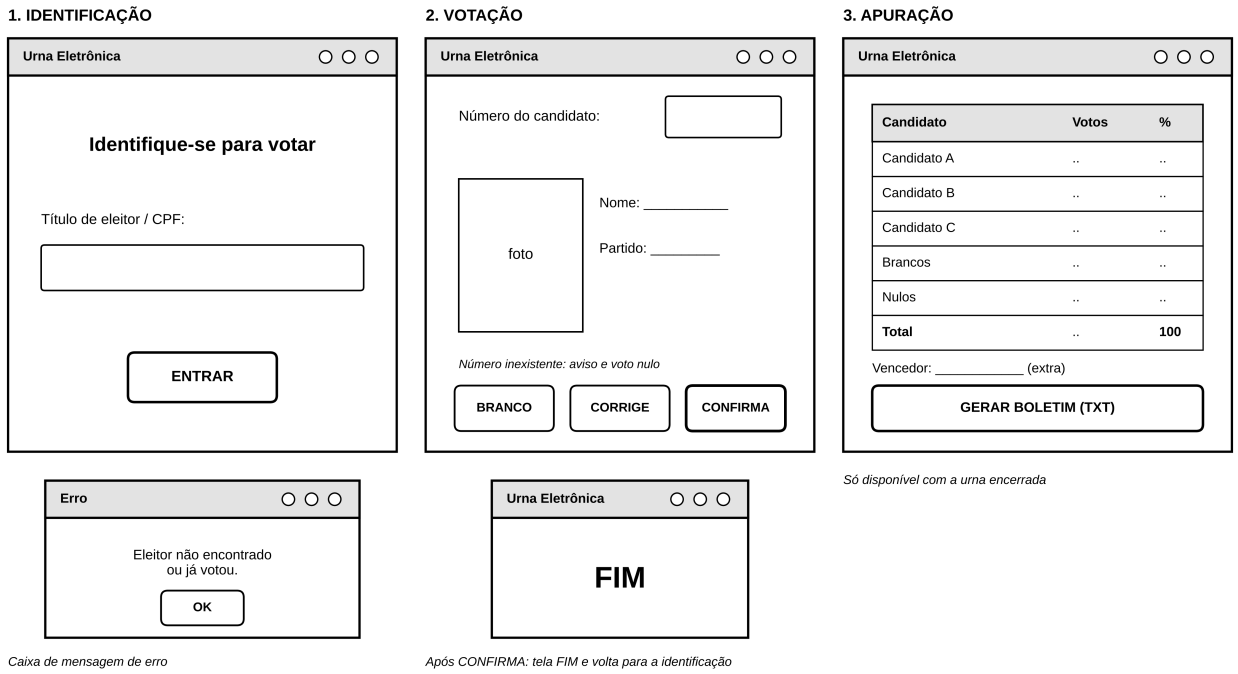


Figura 2 – Esboço das telas: Identificação, Votação e Apuração.

Descrição das três telas principais:

1. **Identificação.** O eleitor digita o título de eleitor ou CPF e clica em ENTRAR. Se não estiver cadastrado ou já tiver votado, aparece uma caixa de mensagem com o erro ("Eleitor não encontrado ou já votou") e um botão OK.
2. **Votação.** O eleitor digita o número do candidato e a tela mostra a foto, o nome e o partido. O botão CONFIRMA registra o voto, CORRIGE limpa o número e BRANCO vota em branco. Se o número não existe, o sistema avisa e o voto conta como nulo. Após CONFIRMA, aparece a tela "FIM" e o sistema volta para a identificação.
3. **Apuração.** Aparece quando a urna é encerrada e mostra uma tabela com candidato, votos e porcentagem, além de brancos, nulos e total. Também mostra o vencedor (extra) e tem o botão GERAR BOLETIM (TXT), disponível só com a urna encerrada.

Resumo: as telas essenciais são três (Identificação, Votação e Apuração), as mesmas que o enunciado pede. A tela "FIM" que aparece depois do CONFIRMA e a caixa de mensagem de erro não são telas separadas: a primeira é só uma mensagem de encerramento do voto e a segunda é um diálogo que aparece sobre a tela atual.

Onde cada requisito aparece no modelo

Código	Onde aparece no modelo	Entrega
RF2.1	Classes Eleitor e Candidato; tabelas eleitor e candidato	2 (listas) e 3 (banco)
RF2.2	Urna com lista de candidatos; tabelas urna e urna_candidato	2 (listas) e 3 (banco)
RF2.3	Urna.votar(int numero); tela de votação (Confirma)	2 (console) e 4 (tela)
RF2.4	Urna.votarBranco(); TipoVoto BRANCO e NULO; botão Branco	2 (console) e 4 (tela)
RF2.5	Urna.encerrar() e contabilizarVotos()	2
RF2.6	Resultado da contabilização gravado em arquivo TXT	3
RF2.7 (extra)	Tela de apuração	4
RN2.1	Eleitor.jaVotou e EleitorJaVotouException	2
RN2.2	Classe Voto sem eleitor; tabela voto sem eleitor_id	2 e 3
RN2.3	Urna.encerrada e UrnaEncerradaException	2 e 4
RN2.4	Composição Urna e Voto	2
RN2.5	Transação no banco (voto e eleitor.ja_votou)	3
RN2.6	EleitorNaoEncontradoException, CandidatoInvalidoException; tabela urna_candidato	3